# Computer Simulation II: Assignment I

[redacted]

### The Diffusion Equation

The equation that describes 1-D diffusion is represented as,

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2}$$

In this equation, $u(x,t)$ signifies the concentration of the substance that is diffusing at a specific location x and time t. The diffusion coefficient, denoted by D, is a positive constant that is determined by the characteristics of the medium. We will utilize various numerical methods to solve this diffusion equation. The initial condition for this solution will be a half-wave, and we will establish boundary conditions either side of the region we are focused on.

### Explicit and Implicit Algorithms

The explicit algorithm for solving the diffusion equation was provided in *diffusion.py*, this numerical method relies on solving differential equations with the function value at a certain time being explicitly expressed in terms of the function value at previous times. Although a straightforward method, they are unstable if the parameters are not chosen carefully, and it was significantly slower then the other algorithms that will be discussed. The provided script was altered to perform the integration by the implicit algorithm of the form $M \cdot \mu_{i+1} = N \cdot \mu_i = b$. The system of linear equations can be solved by LU decomposition for the existing half wave initial conditions. Only a single LU decomposition is required due to the time independence of the matrix. By varying $k(\Delta t)$ the stability of the implicit algorithm was found to be stable in comparison to the explicit, tolerating values much higher than $k = 0.02$ whereas the explicit algorithm collapses beyond that margin. Through the use of LU decomposition the implicit method was the fastest algorithm tested at $\sim 1.5$ seconds per run. The results for each method shown in Figure 1.

### Conjugate Gradient Function

A conjugate gradient function was employed to solve the diffusion equation under the same initial conditions. It is suited for systems of equations whose matrix is symmetric and positive-definite and is often utilised as an iterative algorithm. The time taken was for this method was found to be slower then the LU decomposition method at $\sim 3$ seconds per run. Despite not outperforming the implicit method it performed quickly and it was stable.

### Sparse Matrix Conjugation Gradient Solver

A sparse matrix conjugate gradient solver *cg* was taken from the Scipy Sparse Linalg Library. The input numpy array was converted to a sparse array format using *csc_matrix*. Sparse matrices are mostly made up of zeros and come with the advantage that storing and manipulating them is very efficient for large matrices that would be impractical to handle otherwise. Despite this advantage it was the second slowest at $\sim 7$ seconds per run. A surprising result, possible due it not being as suited for the given matrix parameter size of 500, or a poor implementation. The *cg* conjugation gradient solver is compared over varying matrix size against the implicit algorithm in Figure 2. The use of the very direct LU decomposition approach was likely more effective for the given parameters.

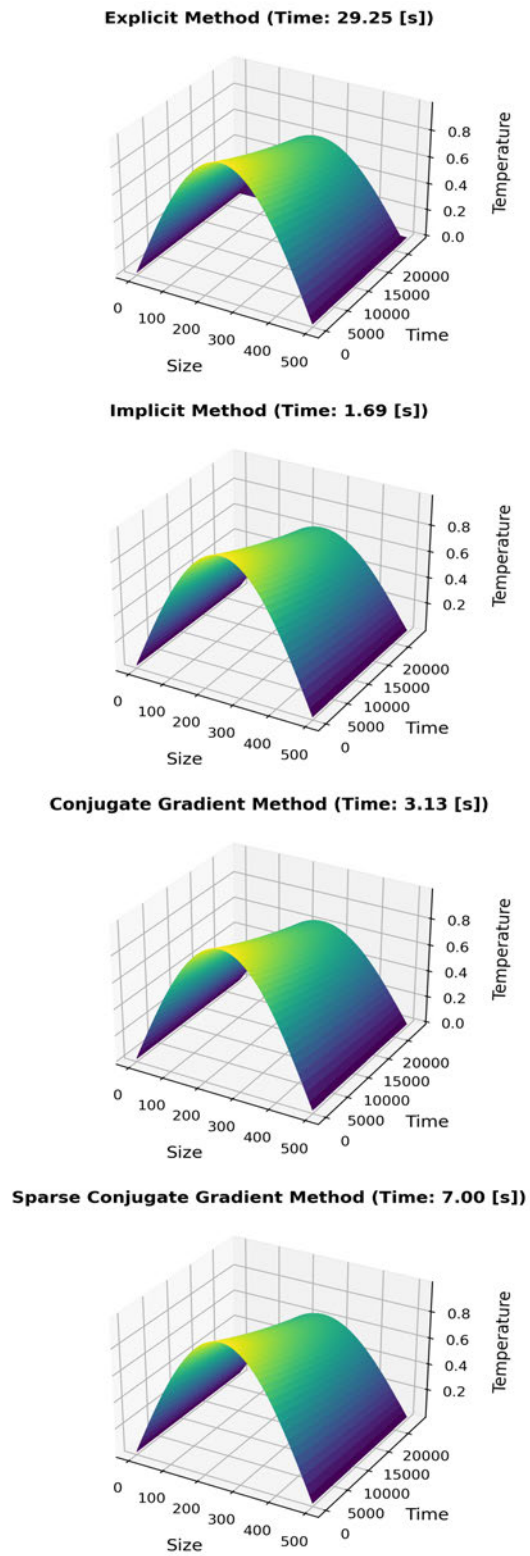


Figure 1: Different numerical algorithms for solving the $1-D$ diffusion equation under half wave initial conditions and fixed boundary conditions. Time in seconds $[s]$ is shown in respective subplot titles.

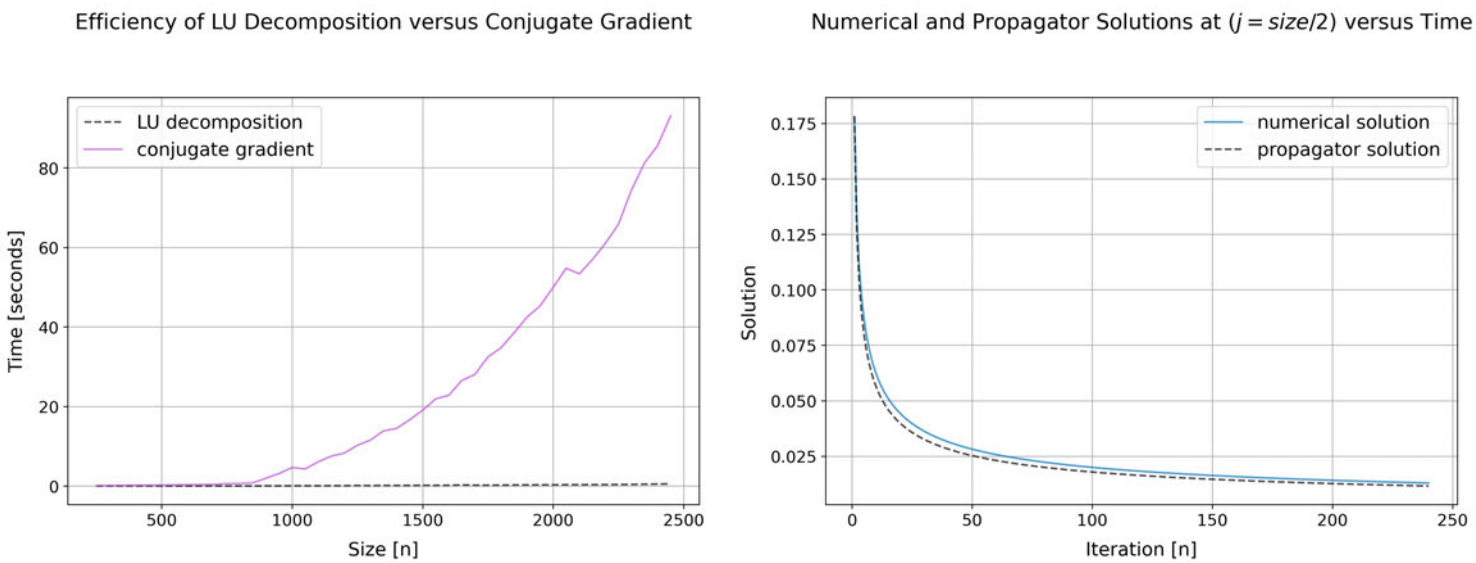


Figure 2: Comparison of LU decomposition (black) time efficiency against conjugate gradient (pink) by matrix size [Left].

Using Gaussian initial conditions, the numerical (blue) versus propagator (black) solutions for the diffusion equation over 240 iterations [Right].

### LU Decomposition versus Conjugate Gradient Efficiency

The efficiency of the *cg* conjugate gradient method was compared to the implicit algorithm as the grid size was increased, shown in Figure 2 [Left]. Beginning at a size parameter of 250 and increasing the system size by 50 each iteration up to $size = 2500$. The decision to increase the number of system size samples was made to improve the resolution of the graph. The scaling of each method is shown with the conjugate gradient solver performing great until $size \sim 1000$, after which the runtime scaling increases quadratically. The implicit method remains at a constant $\sim 1.5$ second runtime throughout. The direct approach of the implicit method with the use of LU decomposition largely removing the need for iterating makes it an extremely effective algorithm for our given scenario.

### Solutions for Diffusion with Gaussian Initial Conditions

The initial conditions of the diffusion was altered to that of a Gaussian located at the centre of the grid space as given by,

$$u[j] = \exp\left[-4000 \cdot (j - \text{size}/2) \cdot (j - \text{size}/2)\,/\text{size}/\text{size}\right].$$

The grid dimension size was then set to 1000. The numerical solution is compared to the propagator solution above as a function of time in Figure 2 [Right], the chosen numerical method was the implicit method due to it being the most efficient for our given parameters in this diffusion scenario. The propagator equation, via Green's function, describes the evolution of the diffusion in this problem. It sharpens to a Dirac delta at $t - t' = 0$ at the point impulse before modelling the spreading of the initial condition. The time at which Gaussian will have spread to this extent was evaluated and this initial time was used in to find the propagator result. The Gaussian was also normalised as the peak value declines in time as the Gaussian spreads as can be seen from the graph. The numerical solution and the propagator solutions are seen to start out at a narrow peak and decline with the rate of diffusion slowing as the matter spreads out. The two equations should both have a similar shape due to the nature of the Gaussian spreading and the propagator equation modeling the evolution of the Gaussian initial conditions which can be seen in figure 2.